Romano's Pizza Group

Sales Behaviour Analysis

1. Project Overview

This project analyses pizza sales behaviour using transactional data from 48,620 orders across various pizza categories, sizes and time periods. The goal is to uncover insights into revenue patterns, peak trading hours, product performance and customer ordering behaviour to guide strategic business and operational decisions for Romano's Pizza Group.

2. Dataset Summary

- Rows: 48,620
- Columns: 15 (after cleaning)
- Key Features:
    - Order details (pizza_id, order_id, quantity, unit_price, total_price)
    - Product details (pizza_name, pizza_category, pizza_size)
    - Time details (order_date, order_time, month, month_name, day_of_week, quarter, hour)
- Missing Data: Zero null values after cleaning

3. Exploratory Data Analysis using Python

We began with data preparation and cleaning in Python using Google Colab:

- Data Loading: Imported the dataset using pandas

- Initial Exploration: Used df.info() to check structure and df.describe(include="all") for summary statistics

df.describe(include = “all”)

```
df.describe(include="all")
```

	pizza_id	order_id	quantity	order_date	order_time	unit_price	total_price	pizza_size	pizza_category	pizza_name	month	month_name	day_of_week	quarter	hour
count	48620.000000	48620.000000	48620.000000	48620	48620	48620.000000	48620.000000	48620	48620	48620	48620.000000	48620	48620	48620.000000	48620.000000
unique	NaN	NaN	NaN	358	16382	NaN	NaN	5	4	32	NaN	12	7	NaN	NaN
top	NaN	NaN	NaN	2015-11-26	12:32:00	NaN	NaN	L	Classic	The Classic Deluxe Pizza	NaN	January	Friday	NaN	NaN
freq	NaN	NaN	NaN	261	26	NaN	NaN	18526	14579	2416	NaN	4288	7723	NaN	NaN
mean	24310.500000	10701.479761	1.019622	NaN	NaN	16.494132	16.821474	NaN	NaN	NaN	6.432476	NaN	NaN	2.484554	15.911806
std	14035.529381	6180.119770	0.143077	NaN	NaN	3.621789	4.437398	NaN	NaN	NaN	3.435386	NaN	NaN	1.116890	3.149836
min	1.000000	1.000000	1.000000	NaN	NaN	9.750000	9.750000	NaN	NaN	NaN	1.000000	NaN	NaN	1.000000	9.000000
25%	12155.750000	5337.000000	1.000000	NaN	NaN	12.750000	12.750000	NaN	NaN	NaN	3.000000	NaN	NaN	1.000000	13.000000
50%	24310.500000	10682.500000	1.000000	NaN	NaN	16.500000	16.500000	NaN	NaN	NaN	6.000000	NaN	NaN	2.000000	16.000000
75%	36465.250000	16100.000000	1.000000	NaN	NaN	20.250000	20.500000	NaN	NaN	NaN	9.000000	NaN	NaN	3.000000	18.000000
max	48620.000000	21350.000000	4.000000	NaN	NaN	35.950000	83.000000	NaN	NaN	NaN	12.000000	NaN	NaN	4.000000	23.000000

df.info() output:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48620 entries, 0 to 48619
Data columns (total 15 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   pizza_id        48620 non-null  int64
 1   order_id        48620 non-null  int64
 2   quantity        48620 non-null  int64
 3   order_date      48620 non-null  object
 4   order_time      48620 non-null  object
 5   unit_price      48620 non-null  float64
 6   total_price     48620 non-null  float64
 7   pizza_size      48620 non-null  object
 8   pizza_category  48620 non-null  object
 9   pizza_name      48620 non-null  object
 10  month           48620 non-null  int64
 11  month_name      48620 non-null  object
 12  day_of_week     48620 non-null  object
 13  quarter         48620 non-null  int64
 14  hour            48620 non-null  int64
dtypes: float64(2), int64(6), object(7)
memory usage: 5.6+ MB
```

- Data Type Issues Found: pizza_id, order_id and quantity were stored as float64 and converted to int64 using .astype(int)

- Mixed Date Formats: order_date column had two different formats mixed together — MM/DD/YYYY (19,587 rows) and DD-MM-YYYY (29,033 rows). Resolved using pd.to_datetime(format='mixed', dayfirst=False)

- Time Column Fix: order_time was stored as plain text. Converted to datetime using pd.to_datetime(format='%H:%M:%S') then extracted the hour column

- Feature Engineering:
  - Created month and month_name columns from order_date
  - Created day_of_week column from order_date
  - Created quarter column from order_date
  - Created hour column from order_time

- Redundant Columns Dropped: pizza_name_id was a duplicate slug of pizza_name and pizza_size combined. pizza_ingredients was not required for any business query. Both were dropped.

- Data Validation: Verified zero null values, zero duplicate rows, and confirmed total_price matched unit_price x quantity across all 48,620 rows

- Database Integration: Connected Python to PostgreSQL via Docker and loaded the cleaned DataFrame into the database for SQL analysis

4. Data Analysis using SQL

We performed structured analysis in PostgreSQL hosted via Docker to answer 11 key business questions:

1. Total Revenue

```
SELECT ROUND(SUM(total_price),2) AS total_revenue FROM pizza_sales;
```

total_revenue
$817,860.05

2. Total Orders

```
SELECT COUNT(DISTINCT order_id) AS total_orders FROM pizza_sales;
```

total_orders
21,350

3. Total Pizzas Sold

```
SELECT SUM(quantity) AS total_pizza_sold FROM pizza_sales;
```

total_pizza_sold
49,574

4. Average Order Value

```
SELECT ROUND(SUM(total_price)/COUNT(DISTINCT order_id),2) AS avg_order_value FROM pizza_sales;
```

avg_order_value
$38.31

5. Revenue by Month

```
SELECT month, month_name, ROUND(SUM(total_price),2) AS total_revenue_monthly
FROM pizza_sales GROUP BY month, month_name ORDER BY month;
```

Month	Month Name	Revenue
1	January	$69,793
2	February	$65,159
3	March	$70,397
4	April	$68,736
5	May	$71,402
6	June	$68,230
7	July	$72,557
8	August	$71,797
9	September	$66,642
10	October	$67,431
11	November	$70,395
12	December	$61,275

6. Orders by Day of Week

```
SELECT day_of_week, COUNT(DISTINCT order_id) AS total_order_by_day_of_week
FROM pizza_sales GROUP BY day_of_week ORDER BY total_order_by_day_of_week DESC;
```

Day of Week	Total Orders
Friday	3,538
Thursday	3,239
Saturday	3,158
Wednesday	3,024
Tuesday	2,973
Monday	2,794
Sunday	2,624

7. Orders by Hour

```
SELECT hour, COUNT(DISTINCT order_id) AS total_order_by_hour
FROM pizza_sales GROUP BY hour ORDER BY hour;
```

Hour	Total Orders
9am	1
10am	60
11am	857
12pm	2,520
1pm	2,455
2pm	778
3pm	794
4pm	1,472
5pm	2,336
6pm	2,399
7pm	2,009
8pm	1,642
9pm	1,198
10pm	663
11pm	192

8. Top 5 Pizzas by Revenue

```
SELECT pizza_name, ROUND(SUM(total_price),2) AS revenue
FROM pizza_sales GROUP BY pizza_name ORDER BY revenue DESC LIMIT 5;
```

Pizza Name	Revenue
The Thai Chicken Pizza	$43,434
The Barbecue Chicken Pizza	$42,768
The California Chicken Pizza	$41,409
The Classic Deluxe Pizza	$38,180
The Spicy Italian Pizza	$34,831

9. Bottom 5 Pizzas by Revenue

```
SELECT pizza_name, ROUND(SUM(total_price),2) AS revenue
FROM pizza_sales GROUP BY pizza_name ORDER BY revenue ASC LIMIT 5;
```

Pizza Name	Revenue
The Brie Carre Pizza	$11,588
The Green Garden Pizza	$13,955
The Spinach Supreme Pizza	$15,277
The Mediterranean Pizza	$15,360
The Spinach Pesto Pizza	$15,596

10. Revenue by Pizza Category

```
SELECT pizza_category, ROUND(SUM(total_price),2) AS total_revenue_pizza_category
FROM pizza_sales GROUP BY pizza_category ORDER BY total_revenue_pizza_category DESC;
```

Category	Revenue
Classic	$220,053
Supreme	$208,197
Chicken	$195,919
Veggie	$193,690

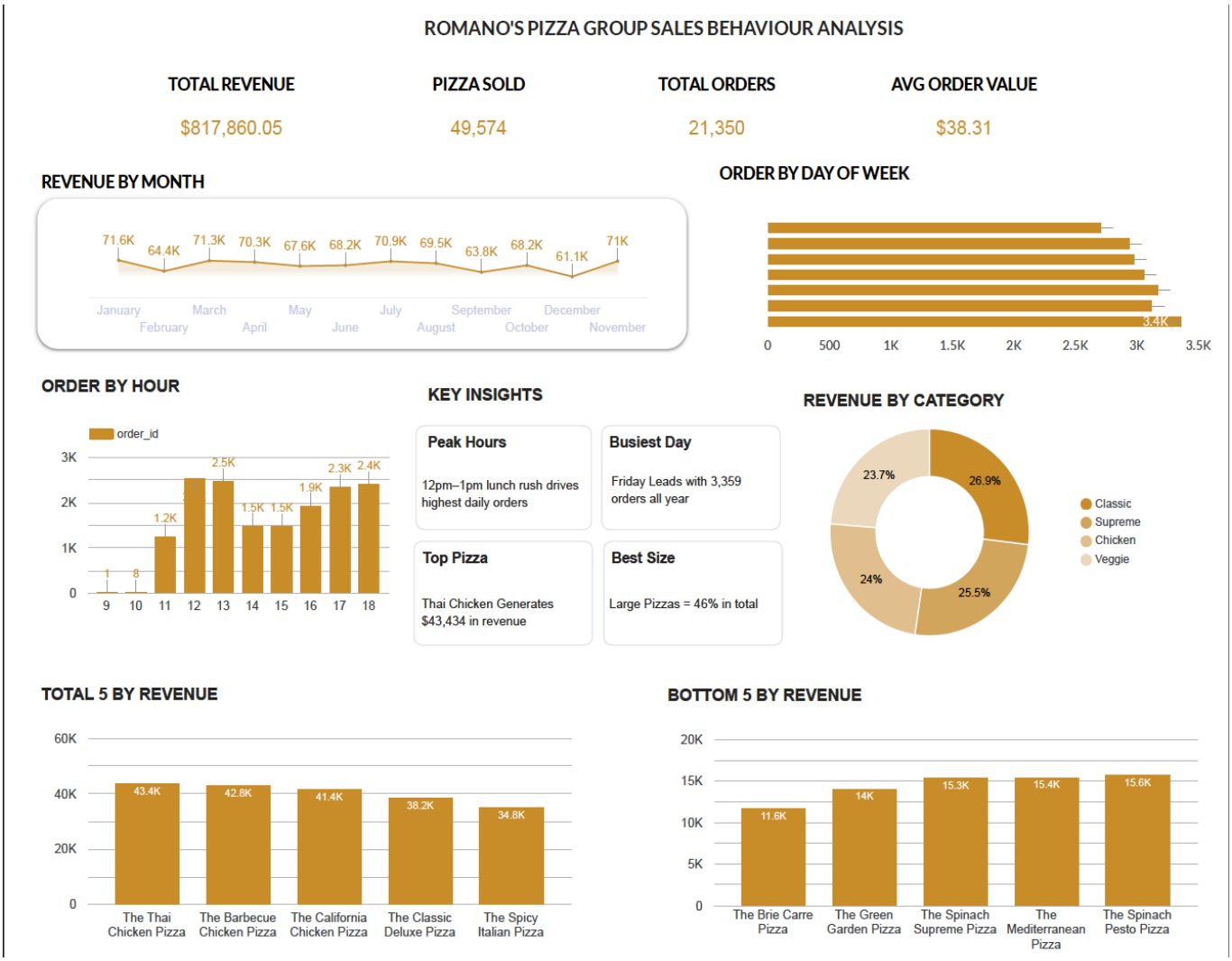
11. Revenue by Pizza Size

```
SELECT pizza_size, ROUND(SUM(total_price),2) AS total_revenue_pizza_size
FROM pizza_sales GROUP BY pizza_size ORDER BY total_revenue_pizza_size DESC;
```

Size	Revenue
L (Large)	$375,318
M (Medium)	$249,382
S (Small)	$178,076
XL	$14,076
XXL	$1,006

5. Customer Behaviour Dashboard in Google Looker Studio

Dashboard Title: Romano's Pizza Group Sales Behaviour Analysis



6. Business Recommendations

- Staff for Peak Hours — Increase staffing during 12pm to 1pm lunch rush and 5pm to 7pm dinner window which drive the highest daily order volumes

- Prioritise Fridays — Friday is the single busiest day with 3,538 orders. Ensure maximum staff availability and ingredient stock every Friday.

- Promote Top Pizzas — The Thai Chicken, BBQ Chicken and California Chicken are the top 3 revenue generators and should be actively featured in promotions and marketing campaigns

- Review Bottom Performers — The Brie Carre Pizza generates only $11,588 for the full year and should be considered for removal or discounted promotion to drive volume

- Remove XXL Size — XXL generates only $1,006 across the entire year suggesting extremely low demand. Removing it simplifies operations and reduces waste

- December Promotions — December records the lowest monthly revenue at $61,275 compared to July peak of $72,557. A targeted promotional campaign in December could bridge this gap

- Focus on Large Size — Large pizzas account for 46% of total revenue. Upselling Medium customers to Large through combo deals could significantly increase average order value

- Invest in Classic Category — Classic pizzas lead all categories at $220,053 and 27% of total revenue. Expanding the Classic range or adding seasonal Classic variants could grow this further